OPIPE-06: Cross-Scope Design Patterns

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 6 of 6 | **Created:** March 2026 | **Last Updated:** 10/15/2026

Correlating Logs, Spans, Metrics, and Events Across Scopes

OpenPipeline's scopes process data independently — but observability value comes from **correlating** across them. A slow API response (span) may be caused by a database timeout (log) that triggers a detected problem (event) and degrades an SLO metric. This notebook covers design patterns that connect the dots across scopes, cascade processing for derived data, and a production readiness checklist.

For individual scope deep-dives, see **OPIPE-02** (spans), **OPIPE-03** (metrics), **OPIPE-05** (events). For log processing, see **OPLOGS-01**. For trace analysis, see **SPANS-04: Service Dependencies & Flow Analysis**.

Table of Contents

1. Shared Dimensions: The Correlation Key
2. Pattern: Same Metric from Multiple Scopes
3. Pattern: Cascade Processing
4. Pattern: Unified Bucket Families
5. When NOT to Use OpenPipeline
6. Production Readiness Checklist
7. Summary
8. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail
Permissions	`storage:logs:read`, `storage:spans:read`, `storage:metrics:read`, `storage:events:read`, `storage:bizevents:read`
Recommended	Complete **OPIPE-01 through OPIPE-05**

1. Shared Dimensions: The Correlation Key

Cross-scope correlation works when different data types share **common dimension values**. If your logs and spans both carry the same service ID and `k8s.namespace.name`, you can join them in DQL.

Correlation Fields Across Scopes

Coverage depends on how each signal was captured, so measure it rather than assume it. One hour on the validation tenant (10/02/2026):

Field	Logs	Spans
`dt.smartscape.service` (`dt.entity.service` is `deprecated`, same coverage)	28,351 of 994,750 — only logs from processes mapped to a service	All 323,480
`dt.service.name`	0	All
`service.name` (OpenTelemetry attribute)	10,920	22,012 — OTel spans only
`k8s.namespace.name`	975,330	304,318
`k8s.deployment.name`	766,504	0

Field	Logs	Spans
`dt.smartscape.host`	Most	244,755 (`dt.entity.host` : 304,662)
`dt.security_context`	If set	If set

Run the same `countIf(isNotNull(...))` shape against your own tenant, and against `metrics` , `events` and `bizevents` , before you design a join on any of these.

> **Dictionary:** `dt.smartscape.service` (`stable`), `dt.entity.service` (`deprecated`), `dt.service.name` (`stable`), `service.name` (`stable`), read 10/02/2026.

The Enrichment Strategy

If a critical dimension is missing from one scope, add it **at the source** or **in OpenPipeline** so that cross-scope queries work:

Scope	Missing field	How to add
Logs	A service key	OneAgent primary tags (`primary_tags.<key>`) set on the process (OPIPE-01) put the same key on every signal from it; for agentless sources, derive it in a DQL processor
Spans	`k8s.deployment.name`	Join on `k8s.namespace.name` plus `k8s.pod.name` instead, or map pod or service to workload with an **Inline lookup** processor
Metrics	`k8s.namespace.name`	Set it as a dimension at the source (extension configuration), or map an existing dimension with an **Inline lookup** processor

The **Inline lookup** processor (SaaS 1.345+, staged rollout from 08/11/2026) maps *"an existing attribute on a record to a new or updated value using a lookup table you define directly in the pipeline"*. Processing *"is based on available records and doesn't take into account record enrichment from external services"*, so the mapping has to come from the record itself or from a table you maintain.

> **Sources:** SaaS 1.345 release notes (DT docs), Processing in OpenPipeline (DT docs).

```
// Cross-scope: Correlate error logs with error spans by service entity
fetch logs, from:-1h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR" and isNotNull(dt.smartscape.service)
| summarize error_logs = count(), by:{dt.smartscape.service}
| lookup [
    fetch spans, from:-1h
    | filter http.response.status_code &gt;= 500 and
isNotNull(dt.smartscape.service)
    | summarize error_spans = count(), by:{dt.smartscape.service}
  ], sourceField:dt.smartscape.service, lookupField:dt.smartscape.service,
fields:{error_spans}
| sort error_logs desc
| limit 10
```

2. Pattern: Same Metric from Multiple Scopes

A powerful validation pattern: extract the **same metric** from two different data sources and compare them. If they diverge, one pipeline has a configuration problem.

Example: Error Count from Logs vs. Spans

Source	Metric Key	Extraction Rule
Logs	`log.error_count`	Count of `status == "ERROR"`, by `dt.smartscape.service`
Spans	`span.error_count`	Count of `http.response.status_code >= 500`, by `dt.smartscape.service`

These metrics should track each other. If logs show 10x more errors than spans, it could mean:
- Application-level errors are logged but not reflected in HTTP status codes
- Span sampling is dropping error spans (see **OPIPE-03** on sampling-aware metrics)
- The log pipeline is catching errors from non-HTTP sources (background jobs, message consumers)

The comparison itself is diagnostic — the divergence tells you something about your data.

```
// Compare: Error counts from logs vs. spans by service
fetch logs, from:-1h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR" and isNotNull(dt.smartscape.service)
| summarize log_errors = count(), by:{dt.smartscape.service}
| lookup [
    fetch spans, from:-1h
    | filter http.response.status_code &gt;= 500 and
isNotNull(dt.smartscape.service)
    | summarize span_errors = count(), by:{dt.smartscape.service}
  ], sourceField:dt.smartscape.service, lookupField:dt.smartscape.service,
fields:{span_errors}
| fieldsAdd ratio = if(isNotNull(span_errors) and span_errors &gt; 0,
    then: round(toDouble(log_errors) / toDouble(span_errors), decimals: 1),
    else: -1.0)
| sort log_errors desc
| limit 10
```

3. Pattern: Cascade Processing

Cascade processing creates a chain of derived data across scopes:

```
Span → extracted business event → business-event metric → SLO
```

Example: Slow Transaction Cascade

Stage	Scope	Configuration	Output
1. Detect	Spans	Filter: `span.kind == "server"` AND `duration > 5s`	Matching spans
2. Extract event	Spans → Business events	Data extraction stage: *Business event* processor with `event.type = "slow_transaction"` (re-ingested into the business-events scope)	Business event per slow span
3. Extract metric	Business events → Metrics	Metric extraction stage in the business-events pipeline: `slow_transaction.count` by `dt.service.name` (copied onto the business event in step 2)	Metric time series

Stage	Scope	Configuration	Output
4. Alert	Metrics	Anomaly detector with a static threshold (for example more than 10 per 5 minutes per service), or an SLO built on the metric	Alert or SLO status

Use a **business event** for step 2, not a Davis event: the Metric extraction stage supports the business-events scope, but not the Davis-events scope, so a Davis event cannot feed step 3.

> **Sources:** Data extraction stage (DT docs), Processing in OpenPipeline (DT docs) — stage table, supported data types per stage.

Design Considerations

- **Each stage adds delay** — a derived record is re-ingested and processed again, so measure the end-to-end delay before you set alert timing on it
- **Failures propagate** — If the span pipeline drops the slow span, the event is never extracted, and the metric never counts it
- **Test end-to-end** — After configuring a cascade, verify data flows through every stage

```
// Identify slow transactions that could trigger a cascade
fetch spans, from:-1h
| filter span.kind == "server" and duration &gt; 5s
| summarize {slow_count = count(), avg_duration_ms = avg(duration / 1ms)},
    by:{dt.service.name}
| sort slow_count desc
| limit 10
```

4. Pattern: Unified Bucket Families

When related data across scopes should share the same lifecycle and access controls, use a **bucket family** naming convention:

Bucket Name	Scope	Contents	Retention
`checkout_logs`	Logs	Checkout service application logs	35 days

Bucket Name	Scope	Contents	Retention
`checkout_spans`	Spans	Checkout service trace data	14 days
`checkout_events`	Events	Checkout-related detected events	35 days
`checkout_bizevents`	Bizevents	Purchase and cart business events	90 days

Benefits

- **Consistent naming** — `checkout_*` makes it obvious which buckets belong together
- **Unified access control** — IAM policies can grant by bucket prefix: the `storage:bucket-name` condition supports `startsWith` (for example `storage:bucket-name startsWith "checkout_"`)
- **Coordinated retention** — Related data expires in a logical sequence (spans first, then logs, business events last)

Querying Across a Bucket Family

```
// Query all buckets in a family to understand data distribution
fetch logs, from:-1h
| filter matchesValue(dt.system.bucket, "checkout_*")
| summarize log_count = count()
| append [
    fetch spans, from:-1h
    | filter matchesValue(dt.system.bucket, "checkout_*")
    | summarize span_count = count()
  ]
```

5. When NOT to Use OpenPipeline

OpenPipeline is powerful but not always the right tool. Prefer DQL query-time processing when:

Scenario	Why Not OpenPipeline	Better Approach
Exploratory analysis	You do not know what patterns you are looking for yet	DQL ad-hoc queries

Scenario	Why Not OpenPipeline	Better Approach
Frequently changing logic	A pipeline change applies only to data ingested after it, and each version leaves differently shaped data behind	DQL with dashboard variables
Complex joins	OpenPipeline cannot join across scopes at ingestion	DQL `lookup` and `join` at query time
One-time investigations	Configuring a pipeline for a single investigation is overhead	DQL `parse` with DPL patterns
Historical data	OpenPipeline only processes new data — it cannot reprocess historical records	DQL for retroactive analysis
Correlation dashboards	Cross-scope correlation requires data from multiple tables	DQL with `append` and `lookup`

The Rule of Thumb

OpenPipeline for permanent, irreversible actions (drop, mask, route, extract). **DQL for everything else.**

If you are unsure whether a transformation belongs in OpenPipeline or DQL, ask: *"Will I regret not having the raw data?"* If yes, keep it raw and process at query time.

6. Production Readiness Checklist

Before deploying a new or modified OpenPipeline configuration to production:

Pipeline Design

- [ ] Each pipeline has a single, clear purpose (one source type per pipeline)
- [ ] Routing rules are ordered from most specific to least specific
- [ ] The default route catches only unmatched data, and you watch its volume
- [ ] No overlapping routing conditions between pipelines

Processing Rules

- [ ] Masking rules applied BEFORE any other processing

- [ ] Drop rules filter noise before extraction (cost efficiency)
- [ ] Parsing rules tested against representative samples
- [ ] Enrichment fields have defined values for all conditions (no nulls)

Metric Extraction

- [ ] Cardinality estimated for each extracted metric (see **OPIPE-04**)
- [ ] No unique-per-record dimensions (`trace.id`, `span.id`, `content`)
- [ ] Sampling-aware extraction for span-derived count metrics (see **OPIPE-03**)
- [ ] Metric keys follow naming convention: `scope.metric_name` (e.g., `span.request_count`)

Security & Governance

- [ ] `dt.security_context` set on all scopes that need access control
- [ ] Security events routed to dedicated long-retention buckets
- [ ] Compliance-relevant data never dropped or sampled
- [ ] PII masking verified on all scopes that handle user data

Testing

- [ ] End-to-end data flow verified: ingestion → pipeline → bucket → query
- [ ] Cascade chains tested: span → event → metric → SLO (if applicable)
- [ ] Volume monitoring in place to detect unexpected spikes or drops
- [ ] Rollback plan documented (revert to previous pipeline configuration)

Summary

In this notebook you learned:

- **Shared dimensions** — Cross-scope correlation requires common fields like `dt.smartscape.service` and `k8s.namespace.name` — measure their coverage per scope first
- **Same metric from multiple scopes** — Extract matching metrics from logs and spans to validate pipeline health

- **Cascade processing** — Chain span → business event → metric → SLO for automated detection and alerting
- **Unified bucket families** — Name related buckets with a common prefix for consistent lifecycle management
- **When NOT to use OpenPipeline** — Keep data raw for exploratory analysis, changing logic, and one-time investigations
- **Production readiness** — Checklist covering pipeline design, processing rules, metric extraction, security, and testing

---

What's Next?

You have completed the OPIPE series. From here:

- **OPLOGS** — Deep-dive into log processing: **OPLOGS-01: OpenPipeline Fundamentals**
- **SPANS** — Master trace analysis: **SPANS-01: Fundamentals**
- **OPMIG** — Migrating from classic logs: **OPMIG-01: OpenPipeline Migration Guide: Part 1**
- **ORGNZ** — Data organization and governance: **ORGNZ-01: Introduction**

References

- OpenPipeline (DT docs)
- Processing in OpenPipeline (DT docs)
- Data extraction stage (DT docs)
- Grail data lakehouse (DT docs)
- DQL cross-data queries (DT docs)
- Service-level objectives (DT docs)
- IAM policy statements (DT docs) — `storage:bucket-name` operators include `startsWith`
- SaaS 1.345 release notes (DT docs) — Inline lookup processor

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.